AWS

AI PRACTITIONER

Complete Exam Revision Notes

Sections 5–12 | GenAI • Bedrock • Prompt Engineering • Amazon Q • ML • SageMaker • Responsible AI • Security

★ Exam Notes Key Concepts Memory Tricks

SECTION 5: Amazon Bedrock & Generative AI

What is Generative AI (GenAI)?

Simple Explanation

Generative AI is AI that can CREATE new content — text, images, audio, video, or code — rather than just analysing existing data. Think of it as a very well-read student who has consumed billions of books and can now write original essays, draw pictures, or answer any question.

Key Concepts

- Creates NEW content (text, images, code, audio, video, 3D)
- Powered by very large models trained on massive datasets
- Uses patterns learned during training to generate outputs
- Foundation Models (FMs) are the base large models that power GenAI
- Examples: ChatGPT (OpenAI), Gemini (Google), Claude (Anthropic)

AWS Context

AWS provides GenAI capabilities primarily through Amazon Bedrock, which gives access to multiple third-party Foundation Models via a single managed API.

Exam-Focused Notes

★ GenAI = AI that GENERATES new content (not just classifies or predicts)

★ Foundation Model (FM) = Large pre-trained model used as a starting point

★ LLM = Large Language Model — a type of FM focused on text

★ Tokens = Units of text that models process (roughly 1 token ≈ 0.75 words)

★ Inference = Using a trained model to generate outputs

🧠 GENERATE = GenAI. If the question mentions creating NEW content, think GenAI / Bedrock.

Amazon Bedrock – Overview

Simple Explanation

Amazon Bedrock is AWS's fully managed service that lets you use and customise powerful AI Foundation Models from multiple providers (Anthropic, Meta, Mistral, AI21, Cohere, Stability AI, Amazon) — all through a single API, without managing any servers.

Key Concepts

- Fully managed — no infrastructure to manage
- Access multiple FMs through one consistent API
- Serverless — pay only for what you use
- Private & secure — your data never used to train third-party models
- Supports fine-tuning, RAG, agents, guardrails

AWS Context

Bedrock is positioned as the core GenAI platform on AWS. It acts as a hub connecting your applications to top AI models while keeping data within your AWS environment.

Example

A company wants to build a customer service chatbot. They use Amazon Bedrock to access Anthropic Claude for conversation, without setting up ML infrastructure.

★ Bedrock = Managed service for accessing MULTIPLE Foundation Models

★ Key benefit: No ML expertise needed; no server management

★ Your data stays PRIVATE — not used to train public models

★ Providers on Bedrock: Amazon Titan, Anthropic Claude, Meta Llama, Mistral, Cohere, AI21, Stability AI

★ Single API = access all models through the same interface

🧠 Think of Bedrock as a 'Model Superstore' — one shop, many AI brands.

Amazon Bedrock – Foundation Model (FM)

Simple Explanation

A Foundation Model is a massive AI model pre-trained on huge amounts of data. It forms the 'foundation' — you can use it as-is or customise it for your specific needs. Like buying a pre-built house and then decorating it to your taste.

Key Concepts

- Pre-trained on massive diverse datasets
- Can be adapted (fine-tuned) for specific tasks
- Multi-modal: text, image, audio, code, video
- Accessed via API — you don't own the weights unless fine-tuned
- Size measured in parameters (billions: 7B, 70B, 405B etc.)

FM Types on Bedrock

Model / Provider	Best For
Amazon Titan	Text, embeddings, multimodal (AWS native)
Anthropic Claude	Complex reasoning, long documents, safe output
Meta Llama 3	Open source, coding, multilingual
Mistral	Efficient, fast, multilingual
Cohere	Enterprise text & embeddings, RAG
AI21 Jurassic	Long-form text generation
Stability AI	Image generation (Stable Diffusion)

★ FM = Pre-trained large model used as a starting point

★ Embeddings = Numerical vector representations of text (used in RAG/search)

★ Multi-modal FM = handles text + images + audio etc.

★ Amazon Titan = AWS's own native Foundation Model family

Amazon Bedrock – Fine-Tuning a Model

Simple Explanation

Fine-tuning is teaching a Foundation Model your specific language, style, or domain knowledge. Imagine hiring an expert generalist consultant and then training them in your company's internal processes — they get smarter about YOUR specific context.

Key Concepts

- Provide labelled training data (prompt-completion pairs)
- Model weights are updated to reflect your specific data
- Results in a customised private model copy
- Requires less data and compute than training from scratch
- Two types: Continued Pre-training (unlabelled) and Fine-Tuning (labelled)

Fine-Tuning vs Continued Pre-training

Fine-Tuning	Continued Pre-Training
Uses labelled data (Q&A pairs)	Uses unlabelled raw text
Teaches specific tasks/style	Teaches domain knowledge
e.g. Customer service responses	e.g. Medical textbooks

★ Fine-Tuning = Update model weights with YOUR labelled data

★ Continued Pre-training = Feed domain text (unlabelled) to build domain knowledge

★ Fine-tuning requires labelled prompt-completion pairs in JSONL format

★ Data stored in S3 for fine-tuning jobs

★ Fine-tuned models are PRIVATE to your account

🧠 Fine = Labelled. Pre = Unlabelled. 'Fine art needs labels; raw pre-school doesn't.'

Amazon Bedrock – FM Evaluation

Simple Explanation

FM Evaluation is a feature in Bedrock that lets you systematically test and compare different models to find which one performs best for your specific use case — like test-driving different cars before buying.

Key Concepts

- Automatic evaluation using built-in metrics
- Human evaluation using a workforce (you or Amazon Mechanical Turk)
- Metrics: accuracy, toxicity, robustness, helpfulness
- Compare multiple models side-by-side
- Helps with model selection before deployment

Evaluation Types

Automatic Evaluation	Human Evaluation
Uses algorithm-based scoring	Uses human judges to rate output
Faster, cheaper, scalable	Slower, costlier, more nuanced
Metrics: ROUGE, BERTScore, F1	Thumbs up/down, quality scores

★ FM Evaluation = Compare models before choosing one for production

★ Automatic = Algorithm scores; Human = People rate quality

★ ROUGE = text summarization metric (overlap of n-grams)
★ BERTScore = semantic similarity using BERT embeddings
★ Use FM Evaluation to reduce model selection risk

Amazon Bedrock – RAG & Knowledge Base

Simple Explanation

RAG (Retrieval-Augmented Generation) lets an AI model look up your private documents BEFORE answering a question — like letting a student open their textbook during an exam. It keeps answers accurate and up to date without re-training the model.

Key Concepts

- RAG = Retrieval + Generation combined
- Store documents → convert to embeddings → store in vector database
- At query time: search for relevant chunks → feed to FM → generate answer
- Bedrock Knowledge Bases = managed RAG implementation
- Reduces hallucinations by grounding answers in real data

RAG Flow

Step	What Happens
1. Ingest	Upload docs (PDF, Word, HTML) to S3
2. Chunk & Embed	Split into chunks, convert to vectors (embeddings)
3. Store	Save vectors in vector store (OpenSearch, Aurora, Pinecone)
4. Query	User question → embed → similarity search → retrieve chunks
5. Generate	FM uses retrieved chunks + question → generates grounded answer

★ RAG = Retrieval-Augmented Generation — fetch then generate
★ Knowledge Base = Bedrock's managed RAG service
★ Embeddings = vector representations used for similarity search
★ Vector Store options: OpenSearch Serverless, Aurora, Redis, Pinecone
★ RAG solves: hallucinations, stale knowledge, private data access
★ No model retraining needed — just update your documents

🧠 RAG = 'Read-Answer-Generate'. Model reads your docs first, then generates an answer.

More GenAI Concepts

Key Terms

- Hallucination — Model confidently generates false information
- Temperature — Controls randomness (0 = deterministic, 1 = creative)
- Top-P / Top-K — Controls diversity of token selection
- Context Window — Max tokens the model can 'see' at once
- Prompt — The input/instruction given to the model
- Completion / Response — The output generated by the model
- Tokens — Smallest unit of text the model processes (~0.75 words)
- Embedding — Dense numerical vector representing meaning of text
- Vector Database — Specialised DB for storing and querying embeddings
- Inference — Running a trained model to get predictions/outputs

★ Hallucination = Model produces confident but WRONG information
★ Temperature=0: most predictable; Temperature=1: most creative/random
★ Context window size limits how much conversation history fits
★ Embeddings enable semantic search (meaning-based, not keyword-based)
★ Inference = prediction/generation at runtime (after training)

Amazon Bedrock – GuardRails

Simple Explanation

GuardRails are like content filters and safety nets for your AI app. They automatically block harmful inputs, filter offensive outputs, redact PII, and keep the AI on-topic — like a bouncer checking people at the door.

Key Concepts

- Content filters — block harmful categories (hate, violence, sexual, insults)
- Denied topics — prevent discussion of specified subjects
- Word filters — block specific words or phrases
- PII redaction — automatically mask personal data in outputs
- Grounding checks — verify output is grounded in retrieved context
- Applied to both INPUT (prompts) and OUTPUT (responses)

GuardRails Features

Feature	Purpose
Content Filters	Block violent, sexual, hate, or self-harm content
Denied Topics	Block specific topics (e.g., competitor products)
Word Filters	Block specific words or phrases

PII Redaction	Mask/remove sensitive personal data
Grounding Check	Ensure output is grounded in source documents
Relevance Check	Ensure output is relevant to the user query

★ GuardRails = Safety filters applied to input AND output
★ PII redaction = auto-mask sensitive data (names, emails, SSNs)
★ Denied topics = prevent model from discussing specific subjects
★ Grounding check = verify answer comes from knowledge base, not hallucination
★ GuardRails work independently of which FM is being used

🧠 GuardRails = GUARD the RAILS (input + output). Think airport security — checks both arrivals and departures.

Amazon Bedrock – Agents

Simple Explanation

Bedrock Agents allow AI models to take actions — not just answer questions. They can call APIs, query databases, execute code, and complete multi-step tasks autonomously. Like giving your chatbot hands to actually DO things, not just talk about them.

Key Concepts

- Agents orchestrate multi-step workflows autonomously
- Use Action Groups to call Lambda functions / APIs
- Connect to Knowledge Bases for RAG capabilities
- Chain of Thought (CoT) — model breaks down task into steps
- ReAct pattern: Reason + Act in a loop until task completes

Agents Components

Component	Role
Foundation Model	The 'brain' that reasons and decides next steps
Action Groups	APIs/Lambda functions the agent can call
Knowledge Base	Documents the agent can retrieve from
Session History	Maintains conversation context across turns
Orchestration	Bedrock manages the reasoning + action loop

★ Agents = FM + Actions + Memory + RAG working together
★ Action Groups = API/Lambda calls the agent can execute

★ ReAct = Reasoning + Acting in a loop (think, act, observe, repeat)

★ Use Agents for multi-step tasks like booking, ordering, data retrieval

★ Agents maintain session context automatically

🧠 AGENT = 'Active GenAI that Executes Normally (Takes actions)'. It DOES things!

Amazon Bedrock – CloudWatch Integration

Key Concepts

- Bedrock sends metrics to Amazon CloudWatch automatically
- Monitor: invocation count, latency, token usage, errors
- Model Invocation Logging — log inputs/outputs to CloudWatch Logs or S3
- Set alarms on latency or error thresholds
- Use for cost monitoring, debugging, compliance

★ CloudWatch = observability for Bedrock (metrics, logs, alarms)

★ Model Invocation Logging = record every prompt + response (audit trail)

★ Useful for compliance, debugging, and cost management

Amazon Bedrock – Pricing

Pricing Models

Model	Description
On-Demand	Pay per token (input + output). Best for variable workloads.
Provisioned Throughput	Reserve model capacity for guaranteed performance. Best for consistent high volume.
Batch Inference	Process large batches of requests at lower cost. Best for non-real-time workloads.

★ On-Demand = Pay per token, no commitment

★ Provisioned Throughput = Pre-pay for reserved capacity (for production)

★ Batch Inference = Async processing, cheapest option

★ Tokens = unit of billing (input tokens + output tokens charged separately)

★ Fine-tuned models: pay for storage + inference separately

Amazon Bedrock – AI Stylist (Use Case Example)

The AI Stylist is an example real-world use case demonstrating how Bedrock components work together in a retail scenario:

- Customer uploads a photo and asks for outfit recommendations
- Bedrock Agent orchestrates the flow
- Knowledge Base contains product catalogue with embeddings
- FM (e.g., Claude) generates personalised recommendations
- GuardRails filter offensive content
- Lambda Actions look up live inventory and pricing

★ AI Stylist = example use case combining Agents + RAG + GuardRails + FM

★ Shows how all Bedrock features integrate in a real application

SECTION 6: Prompt Engineering

What is Prompt Engineering?

Simple Explanation

Prompt Engineering is the art and science of writing instructions to an AI model to get the best possible output. It's like knowing exactly HOW to ask a very smart colleague a question so they give you exactly what you need.

Key Concepts

- No code changes — improve results through better instructions
- Critical skill for getting quality outputs from LLMs
- Iterative process — test, refine, test again
- Applies to all GenAI models and use cases

★ Prompt Engineering = Crafting better inputs to get better outputs

★ No model retraining needed — only change the prompt

★ Key components of a prompt: Instruction, Context, Input Data, Output Format

Prompt Performance Optimization

Techniques to Improve Output Quality

- Be specific and detailed — vague prompts = vague answers
- Specify output format (JSON, bullet points, table, markdown)
- Set the persona ('You are an expert AWS architect...')
- Provide context about the audience ('Explain to a 10-year-old...')
- Include examples (few-shot prompting)
- Limit scope ('Answer in 3 bullet points only')

★ Specific > Vague: More detail in prompt = better output
★ Format specification reduces post-processing work
★ Persona assignment improves domain accuracy
★ Negative constraints help ('Do NOT include...')

Prompt Engineering Techniques

Technique	Description
Zero-Shot	Ask directly with no examples. Works for simple tasks.
One-Shot	Provide one example before the task.
Few-Shot	Provide 2–5 examples. Best for consistent formatting.
Chain-of-Thought (CoT)	Ask model to show reasoning step-by-step. Best for maths/logic.
Tree of Thoughts	Explore multiple reasoning paths. Best for complex problems.
ReAct	Reason + Act in a loop. Best for agents with tools.
Role Prompting	Assign a role/persona to the model.
System Prompt	Hidden instruction that sets model behaviour globally.

★ Zero-Shot: No examples | One-Shot: 1 example | Few-Shot: 2–5 examples
★ Chain-of-Thought = 'Think step by step' → better reasoning
★ ReAct = Reasoning + Acting (used in Bedrock Agents)
★ System prompt = Background instructions (role, tone, constraints)
★ Few-shot prompting is the most reliable for consistent output formatting

🧠 Zero→One→Few = Adding more examples like adding gears to a car. More gears = more control.

Prompt Templates

Simple Explanation

Prompt templates are reusable prompt structures with placeholders that get filled in dynamically at runtime. Like a form letter: the structure is fixed, but the specific content changes per user.

- Used in LangChain, Bedrock Prompt Management, and custom code
- Templates ensure consistent style and format across interactions
- Variables allow dynamic personalisation
- Example: 'Summarise the following {document_type}: {content}'

★ Prompt Template = Reusable prompt structure with variable placeholders

★ Bedrock Prompt Management = AWS service to version and manage prompt templates

★ Templates improve consistency, maintainability, and reusability

SECTION 7: Amazon Q – Deep Dive

Amazon Q Business

Simple Explanation

Amazon Q Business is an AI-powered enterprise chatbot you can connect to your company's data sources (SharePoint, S3, Salesforce, Confluence, etc.) to let employees ask questions in natural language and get answers grounded in your internal documents.

Key Concepts

- Connects to 40+ enterprise data sources via connectors
- Uses RAG under the hood — retrieves from your docs, then generates
- Admin controls: permission-aware (users only see docs they can access)
- Supports document upload, web crawling, and custom data connectors
- Built-in guardrails for safe enterprise use

★ Amazon Q Business = Enterprise AI assistant powered by your company's data

★ Permission-aware = respects existing IAM/SSO access controls

★ Uses RAG — grounded in YOUR documents, not just model training data

★ Data connectors: S3, SharePoint, Confluence, Salesforce, Jira, etc.

★ Ideal for HR, IT helpdesk, knowledge management use cases

🧠 Q Business = your company's internal Google with AI. Searches YOUR data.

Amazon Q Apps

Simple Explanation

Amazon Q Apps allows non-technical employees to build simple AI-powered mini applications from natural language descriptions — no coding required. Like creating a form or tool just by describing what you want in plain English.

- Employees describe an app in natural language → Q builds it
- Examples: email generator, meeting summariser, report formatter
- Apps can be shared with colleagues
- Built on top of Amazon Q Business

★ Q Apps = No-code AI app builder for business users

★ Natural language → working mini-app in minutes

★ Extends Q Business for self-service automation

Amazon Q Developer

Simple Explanation

Amazon Q Developer is an AI coding assistant integrated into IDEs (VS Code, JetBrains, etc.) that helps developers write, explain, debug, and refactor code. It's like having a senior developer paired with you 24/7.

Key Features

- Code completion and code generation from comments
- Chat with your codebase — ask questions about your code
- Security scanning — detects vulnerabilities in real time
- Unit test generation
- Code transformation — upgrade Java/Python versions automatically
- AWS-specific: deep knowledge of AWS APIs and services

★ Q Developer = AI coding assistant for developers (VS Code, JetBrains, CLI)

★ Security scanning = detect vulnerabilities while you code

★ Code transformation = auto-upgrade legacy code to new versions

★ Amazon CodeWhisperer is now part of Amazon Q Developer

★ Free tier available; Pro tier adds organisation management

🧠 Q Developer = CoPilot for AWS. Code faster, safer, smarter.

Amazon Q for AWS Services

Amazon Q is embedded across multiple AWS services to provide in-context AI assistance:

AWS Service	Amazon Q Capability
AWS Console	Ask questions about AWS services in natural language
QuickSight	Q in QuickSight = BI queries in natural language (ask your data)
AWS Glue	Explain and generate ETL code
Amazon Connect	AI agent assist for call center agents
Supply Chain	Q for Supply Chain = demand forecasting and supply insights

★ Q in QuickSight = natural language BI queries ('Show me revenue by region last quarter')

★ Q Developer = coding; Q Business = enterprise knowledge; Q in Console = AWS help

★ Amazon Q is AWS's unifying AI assistant brand across services

SECTION 8: AI & Machine Learning

AI, ML, Deep Learning, and GenAI

These are nested concepts — each is a subset of the one above:

Term	Definition
Artificial Intelligence (AI)	Broad field of machines simulating human intelligence
Machine Learning (ML)	Subset of AI where machines learn from data without explicit programming
Deep Learning (DL)	Subset of ML using neural networks with many layers

Generative AI (GenAI)	Subset of DL that creates new content (text, images, etc.)

★ AI ⊃ ML ⊃ Deep Learning ⊃ Generative AI (nested subsets)

★ ML: learns patterns from data; DL: uses deep neural networks

★ GenAI: specifically CREATES new content

🧠 'All Machines Drive Gracefully' = AI > ML > DL > GenAI (outermost to innermost)

ML Terms You May Encounter in the Exam

Term	Meaning
Feature	Input variable/attribute used to make predictions
Label / Target	The output variable being predicted
Training Data	Labelled data used to train the model
Validation Data	Used to tune hyperparameters during training
Test Data	Held-out data to evaluate final model performance
Model	Mathematical function mapping inputs to outputs
Epoch	One full pass through the entire training dataset
Batch Size	Number of samples processed before updating weights
Loss Function	Measures how wrong the model is (e.g., MSE, Cross-entropy)
Gradient Descent	Optimisation algorithm to minimise the loss
Weights/Parameters	Learned values inside the model

★ Feature = input; Label = output; Model learns to map feature → label

★ Training/Validation/Test split: ~70/15/15 or 80/10/10

★ Epoch = full pass through training data; more epochs = more training

Training Data

Simple Explanation

Training data is the historical examples you feed to an ML model so it can learn patterns. Like showing a child thousands of photos of cats and dogs and telling them which is which.

- Quality of training data directly determines model quality
- Requires labelling (supervised) or just collection (unsupervised)
- Issues: bias, noise, imbalance, insufficient volume
- Data augmentation: artificially expand dataset (flip/rotate images)
- Data wrangling/preprocessing: clean, transform, normalise data

★ Garbage in = Garbage out — data quality is critical
★ Imbalanced data → model biased toward majority class
★ Data augmentation = synthetic expansion of training set
★ Feature engineering = creating new features from existing ones

Supervised Learning

Model learns from labelled data (input + correct output pairs). Most common ML type.

Type	Task	Examples
Classification	Predict a category	Spam/not spam, image classification
Regression	Predict a number	House price, temperature forecast

★ Supervised = Labelled data (X, Y pairs). Teacher tells the model the answer.
★ Classification → discrete output (class/category)
★ Regression → continuous output (number)
★ Algorithms: Linear Regression, Decision Tree, Random Forest, SVM, Neural Nets

Unsupervised Learning

Model learns from UNLABELLED data, finding hidden patterns or structures on its own.

Type	Task	Example
Clustering	Group similar items	Customer segmentation
Dimensionality Reduction	Reduce features	PCA for visualisation
Anomaly Detection	Find outliers	Fraud detection
Association	Find co-occurrence rules	Market basket analysis

★ Unsupervised = Unlabelled data. Model discovers patterns itself.

★ Clustering (K-Means): group similar data points

★ Anomaly Detection: identify unusual patterns (fraud, network intrusion)

🧠 Supervised = Teacher present (labels). Unsupervised = Student discovers alone (no labels).

Self-Supervised Learning

Model creates its own labels from unlabelled data. Used to train Foundation Models (LLMs).

- Example: Mask a word in a sentence → model predicts the masked word
- This is how BERT and GPT models are pre-trained
- Allows training on massive unlabelled datasets

★ Self-Supervised = Model generates its OWN labels from unlabelled data

★ How LLMs are trained: next-token prediction or masked-word prediction

★ Bridges supervised and unsupervised learning

Reinforcement Learning

Model learns by trial and error — taking actions, receiving rewards or penalties, and optimising for maximum cumulative reward. Like training a dog with treats.

- Agent: the model making decisions
- Environment: the world the agent interacts with
- Action: what the agent does
- Reward: positive or negative feedback signal
- Policy: the strategy mapping states to actions

★ RL = Trial-and-error learning with rewards and penalties

★ Key components: Agent, Environment, Action, Reward, Policy

★ Used in: game playing, robotics, recommendation optimisation

RLHF (Reinforcement Learning from Human Feedback)

Simple Explanation

RLHF is the key technique used to align LLMs with human preferences. Human raters rank model outputs → this ranking trains a reward model → the LLM is fine-tuned to maximise the reward. This is how ChatGPT and Claude were made safe and helpful.

- Step 1: Collect human preference data (which response is better?)

- Step 2: Train a Reward Model using human rankings
- Step 3: Fine-tune LLM using RL to maximise reward model score

★ RLHF = Uses human rankings to make LLMs more helpful, harmless, honest
★ Key technique behind alignment of modern LLMs (GPT, Claude, Llama)
★ Reward model = trained on human preferences to score responses
★ RLHF combats: harmful outputs, hallucinations, unhelpful responses

🧠 RLHF = 'Reinforcement Learning with Human as the Feedback source'. Humans rate, AI learns.

Model Fit, Bias, and Variance

Problem	Description	Fix
Underfitting	Model too simple — poor on both training and test data	More features, more complex model
Overfitting	Model too complex — memorises training data, fails on new data	Regularisation, more data, dropout
High Bias	Model assumes too much — underfits	Increase model complexity
High Variance	Model too sensitive to training data — overfits	More data, regularisation

★ Underfitting = high bias = too simple. Overfitting = high variance = too complex.
★ Bias-Variance tradeoff: goal is low bias AND low variance
★ Regularisation (L1/L2, Dropout) helps prevent overfitting
★ Train/Validation/Test split helps detect overfitting early

🧠 'Under-BIKE, Over-FIT': Underfitting is lazy (doesn't learn enough), Overfitting memorises (doesn't generalise).

Model Evaluation Metrics

Metric	Use Case	Formula / Notes
Accuracy	Classification (balanced classes)	Correct / Total predictions

Precision	When false positives are costly	TP / (TP + FP)
Recall (Sensitivity)	When false negatives are costly	TP / (TP + FN)
F1 Score	Balance of Precision and Recall	2 × (P × R) / (P + R)
AUC-ROC	Binary classifiers	Area under ROC curve (0.5–1.0)
RMSE / MAE	Regression tasks	Root Mean Square Error / Mean Absolute Error
BLEU	Translation quality	Overlap between predicted and reference text
ROUGE	Summarisation quality	Recall-oriented n-gram overlap
Perplexity	Language model quality	Lower = better at predicting next word

★ Precision = avoid false alarms; Recall = avoid missing real cases

★ F1 = balance of Precision and Recall (use when classes are imbalanced)

★ BLEU = MT quality; ROUGE = summarisation; Perplexity = LM quality

★ Use Recall for medical diagnosis (missing a disease is worse than a false alarm)

🧠 Precision = 'PRecision = PRoblem with false positives'. Recall = 'ReCAll = Really Can't Afford false negatives'.

Machine Learning – Inferencing

Inferencing is using a trained model to make predictions on new data in production.

Type	Description	Use Case
Real-time Inference	Low-latency single predictions on demand	Chatbot, fraud detection
Batch Inference	Process large datasets offline	Monthly report generation
Streaming Inference	Continuous processing of data streams	Real-time sensor monitoring
Edge Inference	Run model on device (IoT)	Smart cameras, mobile apps

★ Inferencing = applying a trained model to new data (production phase)

★ Real-time: low latency, on-demand; Batch: high throughput, scheduled

★ SageMaker Endpoints = managed real-time inference hosting

Phases of a Machine Learning Project

Phase	Activities
1. Business Problem	Define the problem, success criteria, and metrics
2. Data Collection	Gather raw data from relevant sources
3. Data Preprocessing	Clean, normalise, encode, handle missing values
4. Feature Engineering	Select/create relevant features for the model
5. Model Training	Select algorithm, train on training data
6. Model Evaluation	Validate using metrics on held-out test data
7. Model Deployment	Deploy to endpoint/service for production use
8. Monitoring	Track performance, detect drift, retrain as needed

★ Data preprocessing takes the most time in real ML projects (~80%)

★ Feature engineering often determines model success more than algorithm choice

★ Model drift = performance degrades over time as real-world data changes

Hyperparameters

Hyperparameters are settings you configure BEFORE training that control the training process — not learned from data like weights.

Hyperparameter	Effect
Learning Rate	How fast model updates weights (too high = unstable, too low = slow)
Batch Size	Samples per weight update (larger = stable but slow)
Epochs	How many times to pass through training data
Number of Layers / Neurons	Model capacity and complexity
Dropout Rate	Fraction of neurons randomly disabled (prevents overfitting)
Regularisation (L1/L2)	Penalty on large weights to prevent overfitting

★ Hyperparameters = manually set before training (NOT learned from data)

★ Hyperparameter tuning = finding optimal values (SageMaker Automatic Model Tuning)

★ Learning rate is the most critical hyperparameter to tune

When is ML Not Appropriate?

- Rules-based logic is sufficient — simple deterministic if/then rules work fine
- Insufficient data — ML needs enough data to learn patterns
- Interpretability required — regulations demand explainable decisions (lending, healthcare)
- Cost-benefit mismatch — ML overhead outweighs the benefit for simple problems
- Real-time hard constraints — if latency requirements can't be met

> ★ Use traditional programming when: problem is fully deterministic, data is scarce, or explainability is required by law

> ★ ML is NOT always better — pick the right tool for the job

> ★ Rule of thumb: if you can write clear if/else rules, don't use ML

SECTION 9: AWS Managed AI Services

Why AWS Managed Services?

- Pre-built ML models accessible via API — no ML expertise required
- Fully managed — no infrastructure to provision or maintain
- Pay-per-use pricing
- Scalable, reliable, secure
- Integrate easily with other AWS services

> ★ Managed AI Services = Use AI via API without any ML knowledge

> ★ Great for adding AI features to applications quickly

Amazon Comprehend

NLP (Natural Language Processing) service. Understands text.

Feature	Description
Sentiment Analysis	Positive / Negative / Neutral / Mixed
Entity Recognition	People, organisations, dates, locations
Key Phrase Extraction	Important phrases in the text
Language Detection	Identifies the language of the text
PII Detection	Find personal data (emails, SSNs, phone numbers)

Custom Classification	Train custom text classifier on your data
Topic Modelling	Discover themes across a document collection

★ Comprehend = UNDERSTAND text (NLP tasks)

★ Use cases: social media analysis, customer feedback, document processing

★ Comprehend Medical = healthcare-specific entity extraction

🧠 Comprehend = COM-PREHEND = comprehends / understands language.

Amazon Translate

Neural machine translation service — translates text between 75+ languages.

- Real-time and batch translation
- Custom terminology — preserve brand names/technical terms
- Use case: multilingual websites, localisation, customer support

★ Translate = Language TRANSLATION (75+ languages)

★ Custom Terminology = force specific word translations

Amazon Transcribe

Converts speech (audio/video) to text (Speech-to-Text / ASR).

- Real-time streaming transcription or batch file processing
- Speaker diarisation — identify who said what
- Custom vocabulary — domain-specific words
- PII redaction in transcripts
- Transcribe Medical — specialised for clinical/medical audio

★ Transcribe = Speech → Text (ASR)

★ Diarisation = identify different speakers

★ Use case: call centre transcription, meeting notes, subtitles

🧠 TransCRIBE = SCRIBE writes down what is spoken.

Amazon Polly

Converts text to lifelike speech (Text-to-Speech / TTS).

- 40+ languages, 60+ voices
- Neural TTS (NTTS) — most natural sounding voices
- Speech Synthesis Markup Language (SSML) for fine control (pauses, emphasis)
- Lexicons — custom pronunciation of words

★ Polly = Text → Speech (TTS). Think 'Polly the parrot speaks!'

★ SSML = control prosody (speed, pitch, pauses)

★ Use case: audiobooks, accessibility, IVR systems

🧠 Polly = POLLy speaks. Transcribe listens.

Amazon Rekognition

Computer vision service. Analyses images and videos.

Feature	Use Case
Object/Scene Detection	Identify objects, activities, scenes
Face Detection & Analysis	Age, gender, emotion, eye gaze
Face Recognition	Match faces to a collection (identity verification)
Celebrity Recognition	Identify public figures
Text in Images (OCR)	Extract text from signs, documents in images
Content Moderation	Detect explicit/offensive content
Custom Labels	Train custom object detector on your images
Video Analysis	Detect activities and events in video streams

★ Rekognition = Computer Vision (images and video analysis)

★ Face Recognition ≠ Face Detection. Detection: find faces. Recognition: identify who.

★ Custom Labels = train your own object detector without ML expertise

★ Content Moderation = detect NSFW content in user-uploaded media

🧠 ReKognition = ReKognise objects/faces. Computer VISION.

Amazon Lex

Build conversational chatbots and voice assistants. The technology behind Alexa.

- NLU (Natural Language Understanding) — understand intent from text/speech
- ASR (Automatic Speech Recognition) — built-in speech recognition
- Define intents, utterances, slots, and fulfilment
- Integrates with Lambda for business logic
- Deploy to web, mobile, Amazon Connect (call centre)

★ Lex = Build chatbots and voice bots (same tech as Alexa)
★ Intent = what the user wants. Slot = parameter/variable. Utterance = sample phrase.
★ Use Lex + Lambda + Connect for AI-powered call centres

Amazon Personalize

Real-time personalised recommendations — like building a mini Netflix recommendation engine.

- Uses your user behaviour data (clickstream, purchases, ratings)
- No ML expertise needed — just provide data
- Real-time recommendations via API
- Use cases: product recommendations, personalised email, content ranking

★ Personalize = Recommendations engine (like Netflix/Amazon.com recommendations)
★ Needs: user interactions, item metadata, user metadata
★ Returns ranked list of relevant items for a given user in real time

Amazon Textract

Extracts text AND structured data from scanned documents, forms, and tables — beyond simple OCR.

- OCR: extract raw text from images/PDFs
- Forms: extract key-value pairs (Name: John Smith)
- Tables: extract tabular data with structure preserved
- Queries: ask specific questions about a document
- Signatures: detect signatures on documents

★ Textract = Extract structured data from documents (not just raw OCR)
★ Key advantage over basic OCR: understands forms and tables
★ Use case: digitise invoices, tax forms, medical records

🧠 Textract = eXTRACT text and structure from documents.

Amazon Kendra

Intelligent enterprise search powered by ML. Search across your organisation's content using natural language.

- Indexes documents from S3, SharePoint, RDS, websites, Confluence, etc.
- Understands natural language questions — not just keyword matching
- Returns precise answers, not just links
- Relevance tuning and custom document importance

★ Kendra = AI-powered enterprise search (internal knowledge base search)
★ Natural language queries — 'What is our vacation policy?'
★ Used as data source in Amazon Q Business

Amazon Mechanical Turk

Crowdsourcing marketplace for human intelligence tasks (HITs). Connect to a global workforce for tasks AI can't do reliably.

- Use cases: data labelling, image annotation, survey responses, content moderation
- Workers ('Turkers') complete small tasks for micropayments
- Used to create training data for ML models

★ Mechanical Turk = Human labelling workforce (not AI — humans doing tasks)
★ HITs = Human Intelligence Tasks
★ Used to generate labelled training data for ML models

Amazon Augmented AI (A2I)

Adds a human review loop to ML predictions when confidence is low. Automate most predictions; route low-confidence ones to humans.

- Works with Rekognition (content moderation), Textract (document processing)
- Custom ML workflows via SageMaker
- Review workforce: Amazon Mechanical Turk, private teams, or vendor pool

★ A2I = Human-in-the-loop review for low-confidence ML predictions
★ Combines automation (high confidence) with human review (low confidence)
★ Use case: compliance-critical decisions, medical image review

Amazon Comprehend Medical & Transcribe Medical

Service	Capability
Comprehend Medical	Extract medical entities: medications, diagnoses, dosages, procedures from clinical text
Transcribe Medical	Convert clinical speech/dictation to text with medical vocabulary

★ Medical variants = tuned for healthcare terminology (HIPAA eligible)

★ Comprehend Medical: text NLP for clinical notes → structured data

★ Transcribe Medical: doctor dictation → structured clinical text

Amazon's Hardware for AI

Chip	Purpose
AWS Trainium (Trn1)	Custom chip for TRAINING large deep learning models — up to 50% cost savings vs GPU
AWS Inferentia (Inf2)	Custom chip for running INFERENCE on trained models — high throughput, low cost
NVIDIA A100/H100 GPUs	Available on EC2 P4/P5 instances for ML training and inference

★ Trainium = TRAINING chip (Trn1 instances). Cheaper than GPUs for training.

★ Inferentia = INFERENCE chip (Inf2 instances). Optimised for serving models.

★ Trainium/Inferentia = AWS's custom ML silicon for cost savings

🧠 TRAINium = TRAIN. INFERentia = INFER. The names tell you exactly what they do!

SECTION 10: Amazon SageMaker – Deep Dive

Amazon SageMaker AI – Overview

Simple Explanation

Amazon SageMaker is AWS's fully managed platform to build, train, and deploy machine learning models at scale. It removes heavy lifting at every step of the ML workflow — from data prep to production monitoring.

Key Value

- End-to-end ML platform: data → model → deploy → monitor
- Managed infrastructure — AWS handles scaling, patching
- Multiple purpose-built tools for every phase of ML
- Supports custom code (Python/R) and built-in algorithms

★ SageMaker = End-to-end ML platform (not just one tool — an entire ecosystem)
★ Fully managed — you focus on ML, AWS manages servers
★ Key exam tip: Know WHICH SageMaker tool does WHAT

Amazon SageMaker – Data Tools

Tool	Purpose
SageMaker Data Wrangler	Visual data preparation and feature engineering (no-code)
SageMaker Feature Store	Centralised repository to store, share, and reuse ML features
SageMaker Ground Truth	Data labelling service with human annotation workforce
SageMaker Processing	Run data preprocessing scripts at scale

★ Data Wrangler = drag-and-drop data prep (no-code)
★ Feature Store = reusable feature library (train and serve use same features)
★ Ground Truth = labelling jobs (uses A2I for human review)

Amazon SageMaker – Models and Humans

Tool	Purpose
SageMaker Studio	Unified web IDE for ML development (notebooks, pipelines, models)
SageMaker Notebooks	Managed Jupyter notebooks with pre-configured ML environments
SageMaker Training	Managed training jobs (scale to any number of GPUs/CPUs)

SageMaker Automatic Model Tuning	Bayesian hyperparameter search to find optimal settings
SageMaker Autopilot	AutoML — automatically builds, trains, and tunes the best model
SageMaker JumpStart	Pre-trained models and solution templates (one-click deploy)
SageMaker Endpoints	Deploy models as scalable REST API endpoints
SageMaker Batch Transform	Run batch inference jobs on large datasets

★ Autopilot = AutoML (best model selected and trained automatically)

★ JumpStart = pre-trained models ready to deploy (like app store for ML)

★ Automatic Model Tuning = hyperparameter optimisation (Bayesian search)

★ Endpoints = real-time inference; Batch Transform = offline batch inference

 JumpStart = Jump-start your project with pre-built models. Zero to deployment fast.

Amazon SageMaker – Governance

Tool	Purpose
SageMaker Model Cards	Document model purpose, training details, performance, and bias
SageMaker Model Registry	Central catalog to version, approve, and manage ML models
SageMaker Clarify	Detect bias in data and models, explain predictions
SageMaker Model Monitor	Continuously monitor deployed models for data and model drift

★ Clarify = detect BIAS and explain predictions (XAI = Explainable AI)

★ Model Monitor = detect DRIFT (data drift, model quality drift) in production

★ Model Registry = version control and approval workflow for ML models

★ Model Cards = documentation for responsible ML

Amazon SageMaker – Consoles & Pipelines

Tool	Purpose
SageMaker Pipelines	CI/CD for ML — automate end-to-end training workflows
SageMaker Experiments	Track and compare model training runs
SageMaker Debugger	Debug and profile training jobs (catch vanishing gradients etc.)
SageMaker Canvas	No-code ML model building for business users

★ Pipelines = MLOps automation (trigger retrain, evaluate, deploy pipeline)

★ Experiments = track every training run (hyperparams, metrics, artifacts)

★ Canvas = no-code ML for non-technical users (drag and drop)

★ Debugger = diagnose training issues in real time

Amazon SageMaker – Summary

Phase	SageMaker Tool(s)
Data Prep	Data Wrangler, Processing, Ground Truth
Feature Management	Feature Store
Model Training	Training Jobs, Automatic Model Tuning, Autopilot
Model Evaluation	Clarify, Experiments, Debugger
Model Registry	Model Registry, Model Cards
Deployment	Endpoints, Batch Transform
Monitoring	Model Monitor
MLOps	Pipelines
IDE	Studio, Notebooks
No-Code ML	Canvas, JumpStart, Autopilot

Amazon SageMaker – Extra Features

- SageMaker Neo — compile and optimise models for edge devices
- SageMaker Edge Manager — deploy and manage models on edge devices
- SageMaker Serverless Inference — auto-scale to zero, pay per request
- SageMaker Savings Plans — commit to usage for cost savings
- SageMaker HyperPod — resilient training clusters for LLMs

★ Neo = optimise model for edge hardware (IoT, mobile)

★ Serverless Inference = scale to zero (good for infrequent traffic)

★ HyperPod = resilient large-scale training cluster for foundation models

SECTION 11: AI Challenges & Responsibilities

AI Challenges and Responsibilities – Overview

AI brings immense power but also significant risks and responsibilities. AWS expects practitioners to understand ethical, legal, and security challenges.

Challenge Area	Key Concerns
Bias & Fairness	Models may discriminate if trained on biased data
Transparency	Hard to explain how models reach decisions ('black box')
Privacy	Training data and outputs may expose personal information
Security	Prompt injection, model theft, data poisoning attacks
Hallucinations	Models confidently generate false information
Copyright	Models trained on copyrighted data; output may infringe
Environmental	Training large models consumes significant energy

Responsible AI

AWS's Responsible AI Principles

- Fairness — treat all groups equitably
- Explainability — provide clear reasons for model decisions
- Privacy & Security — protect personal data throughout
- Safety — prevent harmful outputs
- Controllability — humans can override and correct AI systems
- Veracity & Robustness — models should be accurate and reliable
- Governance — clear policies for AI oversight

★ Responsible AI = Fairness, Explainability, Privacy, Safety, Controllability

★ SageMaker Clarify = AWS tool for detecting bias and explaining predictions

★ Human-in-the-loop (A2I) = key mechanism for controllability

★ Model Cards = document model fairness and limitations

GenAI Challenges

Challenge	Description & Mitigation
Hallucinations	False confident outputs → Use RAG to ground answers in facts
Toxicity	Harmful, offensive content → Use GuardRails / content filters
Prompt Injection	Malicious prompts override system instructions → Validate inputs
IP/Copyright	Generated text may reproduce copyrighted material → Legal review
Jailbreaking	Users trick model past safety filters → Robust GuardRails
Data Privacy	Prompts may contain PII → PII redaction, data minimisation
Bias Amplification	Model amplifies training data biases → Evaluate and monitor

★ Prompt Injection = attacker embeds malicious instructions in input to hijack model

★ Hallucination mitigation: RAG + grounding checks + GuardRails

★ Jailbreaking = user bypasses safety guidelines through creative prompts

★ GenAI output should ALWAYS be reviewed for high-stakes decisions

Compliance for AI

- GDPR (EU): Right to explanation for automated decisions, data minimisation
- HIPAA (US): PHI protection applies when AI processes medical data
- NIST AI RMF: Framework for managing AI risk
- ISO/IEC 42001: AI Management System standard
- EU AI Act: Risk-based regulation categorising AI by risk level

★ GDPR → right to explanation for automated decisions affecting EU citizens

★ HIPAA → applies to AI processing Protected Health Information (PHI)

★ Compliance requires: documentation, explainability, audit trails, human oversight

Governance for AI

- AI governance = policies, processes, and controls for responsible AI
- Model inventory/registry: track all models in production
- Model Cards: document training data, performance, limitations
- Approval workflows: human sign-off before deploying models
- Access controls: who can invoke, modify, or deploy models
- Audit trails: log all model invocations for accountability

★ Governance tools in AWS: SageMaker Model Registry, Model Cards, Model Monitor
★ Bedrock: Model Invocation Logging → CloudWatch/S3 for audit trail
★ GuardRails = technical governance (what model can/cannot do)

Security and Privacy for AI

Concern	AWS Mitigation
Data in transit	TLS/HTTPS encryption for all API calls
Data at rest	S3/EBS/KMS encryption for training data and model artifacts
Access control	IAM roles and policies for Bedrock/SageMaker
Network isolation	VPC endpoints to keep traffic off public internet
Model privacy	Bedrock: your fine-tuned models are private, data never shared
Audit logging	CloudTrail for API calls, Bedrock invocation logs
PII protection	Comprehend PII detection, GuardRails PII redaction

★ VPC Endpoints = private access to Bedrock/SageMaker without public internet
★ KMS = encryption of training data, model artifacts, outputs
★ CloudTrail = who did what when (audit logs for all AWS API calls)
★ Bedrock: user data NEVER used to train the underlying foundation models

GenAI Security Scoping Matrix

When building GenAI apps, security responsibilities depend on where in the stack you operate:

Layer	Your Responsibility	AWS Responsibility
Infrastructure	Minimal	Compute, storage, network security
Model	Prompt/response security	Model security, training data security
Application	App security, access control, IAM	API security
Data	Your data encryption, PII handling	AWS data plane security

★ Shared responsibility model APPLIES to GenAI — some security is yours, some is AWS's

★ You are always responsible for: your data, IAM, application code, prompt security

★ AWS is responsible for: underlying model security, infrastructure

MLOps

Simple Explanation

MLOps (Machine Learning Operations) applies DevOps principles to ML — automating the build, test, deploy, and monitor lifecycle of ML models, just like CI/CD for software.

MLOps Practice	AWS Tool
Version control for models	SageMaker Model Registry
Automated training pipelines	SageMaker Pipelines
Experiment tracking	SageMaker Experiments
Model monitoring	SageMaker Model Monitor
Infrastructure as Code	CloudFormation, CDK
Container registry	Amazon ECR
Artifact storage	Amazon S3

★ MLOps = DevOps for ML (automate train → evaluate → deploy → monitor)

★ Key challenge: model drift requires automated retraining triggers

★ SageMaker Pipelines = the core MLOps automation tool on AWS

★ Reproducibility: log everything (data version, code, hyperparameters, metrics)

SECTION 12: AWS Security Services & More

IAM Introduction: Users, Groups, Policies

IAM (Identity and Access Management) controls WHO can do WHAT on AWS resources.

Concept	Description
IAM User	Individual identity (person or application) with credentials
IAM Group	Collection of users sharing the same permissions
IAM Policy	JSON document defining permissions (Allow/Deny + Action + Resource)
IAM Role	Temporary identity assumed by AWS services, applications, or users
Root Account	Full admin access — should NOT be used for daily tasks; enable MFA

- ★ Users = individuals; Groups = teams; Roles = services/temporary access
- ★ Least privilege = grant ONLY the permissions required (key principle)
- ★ Root account = lock it down, enable MFA, never use for daily work
- ★ Policy = JSON with Effect (Allow/Deny), Action, Resource, [Condition]

IAM Policies

Policies define permissions. There are two main types:

Type	Attached To	Example
Identity-based	User, Group, Role	Allow user to access S3
Resource-based	Resource (S3 bucket, KMS key)	Allow cross-account access to bucket

- ★ Explicit Deny ALWAYS overrides Allow (most restrictive wins)
- ★ Inline policy = directly embedded in one user/role (non-reusable)
- ★ Managed policy = reusable, can be attached to multiple identities
- ★ IAM Policy simulator = test policies before applying

IAM Roles

Roles are temporary identities assumed by AWS services or external users. No long-term credentials.

- EC2 instance role — allows EC2 to call AWS services without storing keys
- Lambda execution role — Lambda's permissions to call other services
- Cross-account role — assume role in another AWS account
- Service role — e.g., SageMaker role to access S3, ECR

★ Always use IAM Roles for AWS services (never store access keys on instances)
★ Roles use temporary security credentials (STS tokens) — auto-rotate
★ SageMaker/Bedrock need IAM Roles to access S3, KMS, etc.

Amazon S3

Simple Storage Service — object storage for any type of data.

- Stores objects in buckets (globally unique names)
- Highly durable (99.999999999% = 11 nines)
- Versioning — keep all versions of an object
- Bucket policies — resource-based access control
- Block Public Access — prevent accidental public exposure
- Used as primary storage for ML training data, model artifacts, Bedrock data

★ S3 = object storage for ML data (training data, model artifacts, logs)
★ Block Public Access = ALWAYS enable for data security
★ S3 + KMS = encryption at rest for sensitive ML training data
★ S3 Lifecycle Policies = auto-move old data to cheaper storage classes

Amazon S3 – Storage Classes

Class	Use Case	Key Trait
S3 Standard	Active frequently accessed data	Low latency, highest cost
S3 Intelligent-Tiering	Unknown or changing access patterns	Auto-moves between tiers
S3 Standard-IA	Infrequent access, rapid retrieval	Lower storage, retrieval fee
S3 One Zone-IA	Non-critical infrequent data	Single AZ, cheapest IA
S3 Glacier Instant	Archive, ms retrieval	Cheap, immediate access
S3 Glacier Flexible	Archive, minutes–hours retrieval	Very cheap

S3 Glacier Deep Archive	Long-term archive, 12hr retrieval	Cheapest storage

★ Standard = hot data; IA = warm data; Glacier = cold archive data

★ Intelligent-Tiering = use when you don't know access patterns

★ For ML: training data = Standard; old model artifacts = Glacier

Amazon EC2

Elastic Compute Cloud — virtual servers in the cloud. Used for ML training and inference.

Instance Family	Use Case
P-instances (P3, P4, P5)	GPU instances for ML training (NVIDIA)
G-instances (G4, G5)	GPU instances for inference and graphics
Trn1 (Trainium)	Custom AWS training chip, cost-effective LLM training
Inf2 (Inferentia)	Custom AWS inference chip, low-cost serving
C-instances	CPU-intensive workloads
R-instances	Memory-intensive workloads

★ P-instances = ML training with NVIDIA GPUs

★ Trn1 / Inf2 = AWS custom silicon for ML (cheaper than GPU)

★ Use Spot Instances for fault-tolerant ML training (up to 90% cheaper)

AWS Lambda

Serverless compute — run code without managing servers. Triggered by events.

- Pay per invocation + duration
- Used in Bedrock Agents as Action Groups
- Integrates with S3, API Gateway, DynamoDB, SNS, SQS
- 15-minute max execution time

★ Lambda = serverless code execution (event-driven)

★ Bedrock Agents use Lambda for action execution (API calls, DB queries)

★ Lambda execution role = IAM role defining what Lambda can access

Amazon Macie

ML-powered data security service that discovers and protects sensitive data (PII, credentials) in S3.

- Scans S3 buckets for PII and sensitive data
- Alerts when sensitive data is found in wrong locations
- Critical for GDPR/HIPAA compliance

★ Macie = discover PII and sensitive data in S3 using ML
★ Important for AI/ML: ensure training data doesn't contain unlabelled PII

AWS Config

Continuously records AWS resource configurations and evaluates compliance against rules.

- Track configuration history — what changed and when
- Evaluate compliance rules — 'All S3 buckets must have encryption enabled'
- Trigger alerts via SNS or Lambda on rule violations

★ Config = configuration RECORDER and compliance EVALUATOR
★ Use for: drift detection, compliance audit, change management
★ Config ≠ CloudTrail. Config = what is the state. CloudTrail = who changed it.

Amazon Inspector

Automated vulnerability assessment for EC2 instances, containers, and Lambda functions.

- Scans for OS/software vulnerabilities (CVEs)
- Network reachability analysis
- Integrates with AWS Security Hub

★ Inspector = vulnerability scanner for EC2, ECR, Lambda
★ Critical for SageMaker/Bedrock deployments: scan underlying infrastructure

AWS CloudTrail

Records all AWS API calls — who did what, when, from where. The key audit log.

- Logs: user identity, time, source IP, action taken, resources affected
- Management events: control-plane actions (CreateBucket, RunInstances)
- Data events: data-plane actions (GetObject, PutObject)
- Stored in S3; can stream to CloudWatch Logs

★ CloudTrail = audit trail for ALL AWS API calls (who did what when)

★ Default: management events; Data events must be explicitly enabled

★ Bedrock Model Invocation Logging ≠ CloudTrail. CloudTrail = API-level; Invocation logs = prompt/response level

★ CRITICAL: Enable CloudTrail in ALL regions + log file integrity validation

🧠 CloudTRAIL = TRAIL of breadcrumbs for every action taken on AWS.

AWS Artifact

On-demand access to AWS compliance reports and agreements.

- Download SOC reports, PCI DSS, ISO 27001, HIPAA documentation
- Accept/manage legal agreements (BAA for HIPAA)

★ Artifact = compliance DOCUMENTATION portal (not a security tool)

★ Use to download audit reports for your compliance team

★ BAA (Business Associate Agreement) for HIPAA is signed via Artifact

AWS Audit Manager

Continuously collect evidence to prove compliance with frameworks (PCI DSS, HIPAA, GDPR).

- Automates evidence collection from AWS services
- Maps to compliance frameworks automatically
- Generates audit-ready reports

★ Audit Manager = automate COMPLIANCE EVIDENCE collection

★ Artifact = get reports FROM AWS; Audit Manager = build YOUR compliance reports

AWS Trusted Advisor

Automated advisor that checks your AWS account against best practices in 5 areas.

Pillar	Example Check
Cost Optimisation	Idle EC2 instances, underutilised Reserved Instances

Performance	High-utilisation EC2 instances, CloudFront configuration
Security	MFA on root, S3 public access, security groups with open ports
Fault Tolerance	Multi-AZ RDS, EBS snapshots, Route 53 health checks
Service Limits	VPCs, EC2 instances approaching quota limits

★ Trusted Advisor = automated best-practice checker across 5 pillars

★ Business/Enterprise support = full Trusted Advisor checks

★ Free tier: limited to 7 security/performance checks only

VPC & Network Security

Concept	Description
VPC	Virtual Private Cloud — isolated network for your AWS resources
Subnet	Segment within VPC (Public = internet accessible; Private = internal only)
Security Group	Stateful firewall at instance level (allow rules only)
Network ACL	Stateless firewall at subnet level (allow and deny rules)
VPC Endpoint	Private connection to AWS services without internet
PrivateLink	Private connectivity to AWS/partner services

★ SageMaker/Bedrock in VPC = use VPC endpoints to keep traffic private

★ Security Group = stateful (tracks connections); NACL = stateless (checks every packet)

★ Private subnet = no internet gateway; Public subnet = has internet gateway

★ VPC Endpoint types: Interface (PrivateLink) and Gateway (S3/DynamoDB only)

AWS Security Services – Summary

Service	One-Line Summary
IAM	Who can do what on AWS
S3	Object storage (training data, models)

KMS	Key management for encryption
Macie	Find sensitive data (PII) in S3
GuardDuty	Threat detection using ML (network, API anomalies)
Inspector	Vulnerability scanning (EC2, Lambda, containers)
Config	Configuration compliance tracking
CloudTrail	API audit logging
Artifact	Compliance report downloads
Audit Manager	Automate compliance evidence collection
Trusted Advisor	Best practice checks across 5 pillars
Security Hub	Central security findings aggregator

Scenarios for Security

Common exam scenarios and the correct AWS service to use:

Scenario	Answer
Detect threats in your AWS account automatically	Amazon GuardDuty
Find PII in S3 training data	Amazon Macie
Audit who accessed the Bedrock API and when	AWS CloudTrail
Download HIPAA compliance documents from AWS	AWS Artifact
Ensure all S3 buckets are encrypted (policy enforcement)	AWS Config
Scan SageMaker notebook instance for vulnerabilities	Amazon Inspector
Keep Bedrock API traffic off the public internet	VPC Endpoint
Encrypt SageMaker training data at rest	S3 + KMS
Prevent AI model from producing harmful output	Bedrock GuardRails
Get AI recommendations on cost and security	AWS Trusted Advisor
Prove HIPAA compliance to auditors automatically	AWS Audit Manager

★ EXAM TIP: Read scenarios carefully — map the PROBLEM to the SERVICE

★ GuardDuty = threat detection; Inspector = vulnerability scanning; Macie = data discovery

★ CloudTrail ≠ Config ≠ CloudWatch — all are monitoring/logging but for different purposes

★ CloudTrail: API audit | Config: resource compliance | CloudWatch: metrics/performance

🧠 Security services: 'MAGIC-ITA' = Macie, Artifact, GuardDuty, Inspector, Config, IAM, Trusted Advisor, Audit Manager

— End of AWS AI Practitioner Revision Notes —